KTH

DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING
SECOND CYCLE, 30 CREDITS

Stockholm, Sweden 2026

Joint Radar Emitter and Mode Clustering Using a Transformer Encoder Architecture

Markus Swegmark

Supervisor:	TBD
Examiner:	TBD
Host:	TBD

KTH Royal Institute of Technology
School of Electrical Engineering and Computer Science
Master's Programme in Machine Learning

TRITA-EECS-EX-XXXX:XXX

Abstract

This thesis investigates joint clustering of radar emitters and their operating modes from intercepted pulse descriptor word (PDW) streams using a transformer encoder architecture.

Keywords: transformer, deep clustering, electronic intelligence, radar emitter identification, mode recognition, self-supervised learning.

Sammanfattning

Detta examensarbete undersöker gemensam klustring av radarsändare och deras operativa moder från avlyssnade pulsbeskrivande ord (PDW) med hjälp av en transformerbaserad encoderarkitektur.

Nyckelord: transformer, djup klustring, signalspaning, radarsändaridentifiering, modigenkänning, självövervakad inlärning.

Acknowledgments

I would like to thank my supervisor and examiner at KTH, and the host organization for their support throughout this thesis project.

Stockholm, May 2026
Markus Swegmark

Contents

List of Figures

List of Tables

List of Acronyms

ACC clustering accuracy

AMI adjusted mutual information

ARI adjusted Rand index

DEC deep embedded clustering

ELINT electronic intelligence

ESM electronic support measures

EW electronic warfare

FFN feed-forward network

HDBSCAN hierarchical density-based spatial clustering of applications with noise

MHA multi-head attention

NMI normalized mutual information

PDW pulse descriptor word

TOA time of arrival

UMAP uniform manifold approximation and projection

V-measure harmonic mean of homogeneity and completeness

CHAPTER 1

Introduction

1.1 Background and Motivation

Start with story about Ukraine and Information operations (IO).

Electronic warfare (EW) is becoming increasingly important in modern warfare as technological advancements continues in radar technology, and the advancement of computer algorithms, such as machine learning, which can be superior to humans at some delimited tasks.

One of the main objectives of electronic warfare is to control the electromagnetic spectrum. Identifying radars in your surroundings is a crucial part of this. In the subset of EW called electronic support measure (ESM), where one passively listens to the electromagnetic environment to infer and deduce useful information about it, especially information about other agents in it, including mapping which radars are present and information about them.

In modern electronic warfare, so called agile radars, which can be anything from missiles to airplanes to stationary radars, are becoming more and more sophisticated. When emitting a signal for detection, they are intentionally varying the characteristics of the signal, like frequency, pulse width, amplitude, pulse repetition interval to hide their presence in an increasingly more unpredictable way and in expanding ranges, enabled by modern hardware and evolving algorithms (including machine learning).

This leads to a "cat and mouse"-like situation, where the agile radar agent wants to either scan an area, or lock into a target, sometimes being the entity with the ESM, or something else, without being detected themselves, or in other ways try and confuse the ESM.

Apart from having different algorithms for varying the pulse signal, the agile radars also send out different signals depending on the objective, for example if it's scanning it might have a radar rotating around an axis, leading to an oscillating received signal, while if the radar wants to track a single object it can send a more direct signals, leading to a constant signal in the naïve case. Usually, the algorithms for varying the signal characteristic are pretty simple, maybe it changes frequency in a set sequence or random etc, and keeps it like this. However, this can will make it hard to identify and also more advanced algorithms of course exists. This algorithm configuration for varying the signal characteristics combined is what we refer to a mode of the radar (this includes if it's searching or locking on). If we know the mode of a radar, we basically know how the signal will look like (plus minus some noise and interference etc), and also it tells us about the agent sending out the signal if it's searching or locking in, and also can be used to distinguish between emitters and tell us of how advanced the radar is and perhaps even the goal of the agent. Because of this, it would be ideal to identify not only what signals comes from what radar, but also what modes they are using. Note that two radars can have same modes, but also change the modes. We might need to make some limitations and assume that one radar stays in the same mode. Our input data also give us angle of incoming signal which should enable us to filter or give support for classifying different radars from positioning.

In the field of ES we usually call it radar classification (RC) if we want to classify what type of emitter the pulses are coming from. E.i SA-6 radar, drone XXY, etc. While if we want to say exactly what instance it is, that is also distiguish the number of emitters from each emitter as well, e.i SA-6 radar 3, drone XXY 1 and 3, we call that specific radar identification (SRI). So you can say that RC is a superset of SR, and since it provied more informaiton it is more advantageous - however the marginal strategic gain from information of SRI compared to RC can be discuess while the marginal cost might be discussed. The marginal gains is we know the number of emitters exactly, and the ID hopefully, however this problem is sometimes harder and might have lower accurary compared to pure RC, thus decreaseing the Marginal gains. The marginal cost is however that out model might get more complicated, and depending on our model, it could also decrease the accuracy of the classification problem - we will not allow this to happened.

1.2 Problem Statement

Modern radar systems are evolving rapidly. They can change their transmission behavior on the fly (often called “agile” radars), which makes it increasingly difficult for traditional, rule-based methods to classify and identify them reliably. In Electronic Warfare (EW), Electronic Support Measures (ESM) systems passively listen to the electromagnetic environment and aim to detect, characterize, and identify radar emitters in order to enhance situational awareness and protection. Radars emit short bursts of energy known as pulses. Different radars tend to produce characteristic patterns across pulse features such as carrier frequency, pulse width, time of arrival, and angle of arrival. By analyzing these patterns, it is possible to infer how many emitters are present and which emitters they are. However, overlapping signals, agile behavior, noise, and dynamic scenarios make this a complex pattern recognition problem.

This thesis addresses the end-to-end ML classification problem of determining, from recorded radar pulses, both the number of radar emitters present in a flight simulation and their identities. The work will use labeled, simulator-generated ESM data provided by Saab, in which each pulse is described by features such as frequency, pulse width, time of arrival (ToA), and angle of arrival (AoA). The problem is formulated as a supervised learning task, with the goal of training models that map pulse observations to emitter identities. For this thesis you will both design an AI model but also analyze the simulated data. To understand how good the model is it is also required to understand the difficulty of the data it is evaluated on. You will create metrics for measuring dataset complexity and then compare how your model performs on data of varying complexity according to your metrics.

1.3 Research Questions

The thesis addresses the following research questions:

RQ1 Placeholder research question.

RQ2 Placeholder research question.

1.4 Objectives and Scope

Placeholder paragraph.

1.5 Contributions

The main contributions of this thesis are:

- Placeholder contribution.
- Placeholder contribution.

1.6 Ethical and Societal Considerations

Placeholder paragraph.

1.7 Thesis Outline

The remainder of this thesis is organized as follows. Chapter 2 introduces the necessary background on radar signal processing and deep learning. Chapter 3 surveys related work. Chapter 4 describes the proposed method. Chapter 5 presents experiments and results. Chapter 6 discusses the findings. Chapter 7 concludes and outlines future work.

CHAPTER 2

Background

2.1 Radar Systems and Passive Reception

Radio is the art of communicating using electromagnetic waves, traditionally done, but not limited to, radio waves. Discovered in theory in the electromagnetic revolution in the 1800s by the Maxwell and others, and made reality in the end of that centrury by Guglielmo Marconi who in 1891 who transmitted the first morse code over a kilometer using radio waves from his device. xt

Radar, which is an abbreviation for radio detection and range, is a form of echolocalisation, using radio waves. It was discovered in the early 1900s, and it's development was greatly boosted due to it's utility detecting plans and warships in world war 2.

2.2 Radar Emitters and Operating Modes

Placeholder paragraph.

2.3 Pulse Descriptor Words

Placeholder paragraph.

2.3.1 EW Signal Processing Pipeline

In EW and ESM contexts, the path from the electromagnetic environment to downstream reasoning is often described as a *signal processing pipeline.* At a high level, wideband receivers capture emissions, detectors mark pulse events, and estimators extract per-pulse parameters such as carrier frequency, pulse width, and time of arrival. Deinterleaving then groups pulses into coherent trains and emits PDWs (or equivalent feature vectors) as the interface to higher-level ELINT tasks, including emitter and mode analysis. Placeholder paragraph.

2.4 Machine Learning Fundamentals

Placeholder paragraph.

2.5 Clustering Methods

Placeholder paragraph.

2.6 Deep Representation Learning

Placeholder paragraph.

2.7 The Transformer Encoder

Placeholder paragraph.

CHAPTER 3

Related Work

3.1 Classical Radar Emitter Identification

Placeholder paragraph.

3.2 Deep Learning for Radar Signal Analysis

Placeholder paragraph.

3.3 Deep Clustering

Placeholder paragraph.

3.4 Transformers for Time Series and Sets

Placeholder paragraph.

3.5 Joint and Multi-Task Clustering

Placeholder paragraph.

CHAPTER 4

Method

4.1 Problem Formulation

This chapter studies unsupervised structure recovery from a time-ordered stream of PDWs produced by an ESM or deinterleaving front end, as outlined in Section 2.3.1. The goal is twofold: associate each pulse with a physical *emitter instance* and, within that instance, with an *operating mode*, without prior knowledge of how many emitters or modes appear in the data. Both quantities are latent because labels are scarce in ELINT settings and because novel emitters and modes must be accommodated.

Observed input

Let a sequence of N pulses be represented by feature vectors

$$\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N), \qquad \boldsymbol{x}_n \in \mathbb{R}^d, \tag{4.1}$$

where each $\boldsymbol{x}_n$ encodes the measured pulse parameters associated with one PDW (carrier frequency, pulse width, timing, spatial cues, and any auxiliary fields). The sequence may contain irregular sampling, missing pulses, and spurious detections inherited from upstream processing; robustness to such effects is a design constraint rather than part of the formal specification.

Latent emitter and mode assignments

For each index $n \in \{1, \ldots, N\}$, let e_n denote the emitter identity responsible for pulse n and let m_n denote the operating mode within that emitter. These variables take values in finite label sets whose cardinalities

$$K_{\mathrm{em}} = |\{e_1, \ldots, e_N\}|, \qquad K_{\mathrm{md}} = |\{m_1, \ldots, m_N\}| \tag{4.2}$$

are *unknown a priori* and need not match the number of physical platforms in the environment (for example, co-located antennas may map to a single inferred emitter index, or a single platform may split across indices when observations are ambiguous). The pair (e_n, m_n) is not observed at training time; the learning problem is to infer representations from which both clusterings can be recovered reliably.

Desired outputs: dual embeddings and discrete labels

The parametric model considered in this thesis maps each input sequence to two parallel trajectories in Euclidean embedding spaces,

$$\boldsymbol{Z}^{\mathrm{em}} = (\boldsymbol{z}_1^{\mathrm{em}}, \ldots, \boldsymbol{z}_N^{\mathrm{em}}), \quad \boldsymbol{z}_n^{\mathrm{em}} \in \mathbb{R}^p, \qquad \boldsymbol{Z}^{\mathrm{md}} = (\boldsymbol{z}_1^{\mathrm{md}}, \ldots, \boldsymbol{z}_N^{\mathrm{md}}), \quad \boldsymbol{z}_n^{\mathrm{md}} \in \mathbb{R}^q, \tag{4.3}$$

with dimensions p and q fixed by the architecture. Discrete emitter labels $\hat{e}_n$ and mode labels $\hat{m}_n$ are obtained *post hoc* by clustering $\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^N$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}_{n=1}^N$, or by reading off prototype assignments when the training objective includes cluster parameters. Thus the immediate trainable targets are the continuous vectors in Equation (4.3); hard partitions are induced indirectly.

The two trajectories serve different roles. Emitter-oriented vectors $\boldsymbol{z}_n^{\mathrm{em}}$ are primarily a vehicle for estimating consistent emitter membership; their main deliverable is a per-pulse emitter index suitable for downstream ELINT workflows such as emitter counting and localisation. Mode-oriented vectors $\boldsymbol{z}_n^{\mathrm{md}}$ additionally aim to summarise mode-dependent waveform behaviour so that operating modes can be inspected, compared, or matched against a library of known modes once such references become available.

Architecturally, both streams are produced from a shared encoder $f_{\boldsymbol{\theta}}$ with task-specific heads (see Section 4.3); in deployment, emitter embeddings may be less critical to retain after discrete $\hat{e}_n$ have been fixed, whereas mode embeddings (or intermediate encoder activations feeding the mode head) are natural inputs to specialised mode classifiers.

Learning objective

The optimisation problem couples representation learning with clustering-friendly losses so that the geometry of $\boldsymbol{Z}^{\mathrm{em}}$ and $\boldsymbol{Z}^{\mathrm{md}}$ aligns with emitter and mode structure, respectively. The concrete loss terms and training protocol are given in Sections 4.4 and 4.5.

4.2 Data Representation and Preprocessing

ELINT data are difficult to obtain at scale, and operational recordings are often sensitive. This work therefore relies on synthetic pulse trains from a proprietary scenario simulator (Saab), which yields controlled scenarios and full supervision of emitter identity and operating mode for evaluation, while the learning objectives remain unsupervised with respect to those labels.

Each scenario is a time-ordered sequence of PDWs together with emitter and mode annotations. Scenarios vary in the number of emitters and in sequence length so that training covers diverse traffic densities and temporal horizons; exact counts, train/validation/test splits, and sampling ranges are reported in Chapter 5.

Windowing and context length

The encoder operates on fixed-length windows because self-attention cost scales quadratically with sequence length. Let L denote the window length (in pulses) fed to the model; experiments use $L = 1024$. Each scenario is partitioned into contiguous windows of length L. If the remainder after the last full window is shorter than L, it is dropped in the present pipeline; padding with an attention mask would retain the tail without changing the architecture and is left as a straightforward extension.

Training windows are extracted with a stride of $L/2$, so consecutive windows overlap by half their length. This increases the number of training pulses seen per scenario and reduces sensitivity to the particular alignment of window boundaries relative to mode transitions. Validation and test windows use stride L (no overlap), so that metrics are not inflated by near-duplicate windows.

Per-feature scaling

Amplitude is already provided on a logarithmic scale (decibels). Continuous fields are transformed before windowing so that statistics are computed on the training split only and then applied to validation and test data. The log-amplitude, carrier frequency, and pulse width are standardised to zero mean and unit variance using the training-set mean and standard deviation of each field. Each TOA is first min–max scaled to $[0, 1]$ using training-set bounds; within every window, all rescaled TOA values are shifted by subtracting the TOA of the first pulse so that the window starts at time zero. This removes absolute clock offset while preserving intra-window timing structure.

Windows are grouped into mini-batches for optimisation; batch size and per-epoch shuffling of windows are stated in Chapter 5.

4.3 Transformer Encoder Architecture

The encoder $f_{\boldsymbol{\theta}}$ takes the form of a transformer-encoder backbone [1] with a shared trunk followed by two task-specific branches. This layout mirrors the dual-embedding structure introduced in Section 4.1: the trunk produces a single contextualised representation of the input window, which the branches specialise into emitter-oriented and mode-oriented embeddings.

Input embedding

Each PDW vector $\boldsymbol{x}_n \in \mathbb{R}^d$, normalised as described in Section 4.2, is mapped to a token embedding through an affine projection

$$\boldsymbol{h}_n^{(0)} = \boldsymbol{W}_{\text{in}} \boldsymbol{x}_n + \boldsymbol{b}_{\text{in}}, \qquad \boldsymbol{h}_n^{(0)} \in \mathbb{R}^{d_{\text{model}}}, \tag{4.4}$$

where $\boldsymbol{W}_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times d}$ and d_{model} is the trunk width specified below. No positional encoding is added to $\boldsymbol{h}_n^{(0)}$. The temporal ordering of pulses is already carried by the TOA entry of $\boldsymbol{x}_n$, which makes a separate position signal redundant; consistent with this argument, Gunn et al. [2] report that adding sinusoidal or learnable positional encodings to a transformer deinterleaver yields a small but reproducible drop in performance, attributed to the resulting redundancy with the TOA feature.

Shared trunk

The token sequence $(\boldsymbol{h}_1^{(0)}, \ldots, \boldsymbol{h}_L^{(0)})$ is processed by a stack of N_{sh} standard transformer-encoder layers, each comprising a MHA sub-layer and a position-wise FFN sub-layer with residual connections and layer normalisation around both. Self-attention is unmasked, so every pulse can attend to every other pulse in the window. The trunk operates with hidden dimension $d_{\text{model}} = 256$, $h_{\text{sh}} = 8$ attention heads (so that each head has dimension 32), and an inner FFN dimension of $d_{\text{ff,sh}} = 2048$; $N_{\text{sh}} = 6$ layers are used. Its output is a contextualised token sequence $\boldsymbol{H}^{\text{sh}} = (\boldsymbol{h}_1^{\text{sh}}, \ldots, \boldsymbol{h}_L^{\text{sh}})$ with $\boldsymbol{h}_n^{\text{sh}} \in \mathbb{R}^{256}$, in which information has been mixed across the full window.

Task-specific branches

The trunk output feeds two parallel branches that do not share parameters, producing the dual embeddings of Equation (4.3). Each branch begins with an affine projection that reduces the hidden width from $d_{\text{model}} = 256$ to $d_{\text{br}} = 128$, after which $N_{\text{br}} = 2$ transformer-encoder layers of the same form as the trunk are applied. Within a branch, MHA uses $h_{\text{br}} = 4$ heads (again with per-head dimension 32) and the FFN inner dimension is $d_{\text{ff,br}} = 1024$. The reduced width and depth reflect a deliberate asymmetry: the trunk is intended to carry the heavy lifting of contextual mixing, while the branches only need to specialise the representation to either emitter identity or operating mode. Write the resulting token sequences as $\boldsymbol{H}^{\text{em}} = (\boldsymbol{h}_1^{\text{em}}, \ldots, \boldsymbol{h}_L^{\text{em}})$ and $\boldsymbol{H}^{\text{md}} = (\boldsymbol{h}_1^{\text{md}}, \ldots, \boldsymbol{h}_L^{\text{md}})$ with $\boldsymbol{h}_n^{\text{em}}, \boldsymbol{h}_n^{\text{md}} \in \mathbb{R}^{128}$.

Projection heads

Each branch terminates in a linear projection head that maps every token to its respective embedding space,

$$\boldsymbol{z}_n^{\text{em}} = \boldsymbol{W}_{\text{em}} \boldsymbol{h}_n^{\text{em}} + \boldsymbol{b}_{\text{em}}, \qquad \boldsymbol{z}_n^{\text{md}} = \boldsymbol{W}_{\text{md}} \boldsymbol{h}_n^{\text{md}} + \boldsymbol{b}_{\text{md}}, \tag{4.5}$$

with $\boldsymbol{z}_n^{\text{em}} \in \mathbb{R}^p$ and $\boldsymbol{z}_n^{\text{md}} \in \mathbb{R}^q$ for $p = 8$ and $q = 16$. The mode space is assigned the higher dimensionality because, conditional on emitter identity, mode structure is expected to require a richer geometry to separate subtle waveform regimes, whereas emitter membership induces a coarser partition over the same pulses and is well served by a more compact embedding.

Per-window clustering with HDBSCAN

The projection heads in Equation (4.5) map each pulse to a continuous vector, whereas the latent emitter and mode assignments in Section 4.1 are discrete. HDBSCAN [3] is applied independently to the two embedding trajectories within every analysis window: the L points $\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^{L}$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}_{n=1}^{L}$ each form a separate clustering instance on the corresponding branch. Each window is thus a self-contained pulse train segment in embedding space, consistent with the windowing of Section 4.2, and the effective cardinalities of the inferred emitter and mode partitions follow the density hierarchy instead of fixing K_{em} or K_{md} a priori.

The collection of all linear maps in Equations (4.4) and (4.5) together with the attention and FFN weights of the trunk and branches constitutes the parameter vector $\boldsymbol{\theta}$ that the loss in Section 4.4 optimises.

4.4 Loss Function and Clustering Objective

The trainable objective combines contrastive terms on the emitter and mode branches so that both embedding geometries in Equation (4.3) become informative for the unsupervised clusterings described in Section 4.1. This section summarizes the representation-learning rationale, states a generic normalized temperature-scaled cross-entropy (NT-Xent) building block [4], and explains how it is instantiated separately for emitter-centric deinterleaving and for mode-centric structure.

Contrastive learning viewpoint

Contrastive objectives encourage invariance to nuisance transformations while preserving factors that vary across instances [4]. Given two stochastic *views* of the same underlying input (for example two augmentations of the same PDW window), the model should assign similar embeddings to both views, whereas embeddings of unrelated inputs should disagree. Minimizing the resulting classification-style loss is closely related to maximizing a lower bound on mutual information between representations and predictive targets in contrastive predictive coding [5], and more pragmatically it acts as a metric-learning surrogate that spreads the batch on the hypersphere defined by ℓ_2 normalization. A large set of negatives drawn from the mini-batch is essential: without repulsive pressure on unrelated pairs, encoders can trivially collapse to a constant map and still achieve low training error on a naïve agreement loss [4].

Swapping cluster assignments instead of raw embeddings is an alternative that couples representation learning with discrete structure [6]; the present work stays with embedding-space contrast for clarity of exposition, but the same dual-embedding architecture could be paired with a SwAV-style term if prototype banks were introduced.

NT-Xent building block

Let $\phi(\boldsymbol{z}) = \boldsymbol{z}/\|\boldsymbol{z}\|_2$ denote ℓ_2 normalization and let $\tau > 0$ denote the contrastive temperature. Write the scaled cosine similarity as

$$s_\tau(\boldsymbol{a}, \boldsymbol{b}) = \frac{1}{\tau}\,\phi(\boldsymbol{a})^\top \phi(\boldsymbol{b}). \tag{4.6}$$

The temperature τ rescales inner products after normalization: smaller τ sharpens the softmax in Equation (4.7), so the loss concentrates on the hardest negatives and gradients emphasize fine separation on the sphere, whereas larger τ yields a softer distribution closer to uniform and can stabilize optimization when embeddings are poorly separated early in training [4]. All contrastive terms in this thesis share a fixed $\tau = 0.2$, in line with values often used for NT-Xent on normalized features [4].

For a finite index set $\mathcal{I}$ with embeddings $\{\boldsymbol{z}_i\}_{i\in\mathcal{I}}$ and a distinguished *positive* index $p(i) \neq i$ for each anchor $i \in \mathcal{I}$, define the InfoNCE / NT-Xent contribution

$$\mathcal{L}_{\mathrm{NT}}(\mathcal{I}, \{p(i)\}_{i\in\mathcal{I}}) = -\frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \log \frac{\exp\Big(s_\tau(\boldsymbol{z}_i, \boldsymbol{z}_{p(i)})\Big)}{\sum_{\substack{j\in\mathcal{I}\\ j\neq i}} \exp\Big(s_\tau(\boldsymbol{z}_i, \boldsymbol{z}_j)\Big)}. \tag{4.7}$$

The denominator collects every candidate in the mini-batch except the anchor itself, so unrelated pairs act as negatives and collapse is discouraged [4]. The same expression is reused below with different choices of index sets, corresponding to different geometric constraints on the emitter and mode spaces.

Emitter-oriented term (deinterleaving)

The emitter branch targets a representation in which pulses from the same physical emitter are close and pulses from different emitters are separated, which is the geometric content expected of a deinterleaving embedding [2]. For each window in a mini-batch, two views are formed by independent draws of the augmentation pipeline (once augmentations beyond deterministic scaling are fixed in Section 4.2); the two views share semantics at the pulse level up to controlled perturbations, so they play the role of a positive pair in the sense of SimCLR [4].

Because emitter identity is a *window-level* hypothesis in dense traffic, the contrastive signal is applied to a per-window summary of the emitter tokens rather than to isolated pulses without batch context. Let $b = 1, \ldots, B$ index windows in a batch and let $\bar{\boldsymbol{z}}_b^{\mathrm{em}} \in \mathbb{R}^p$ denote a fixed aggregation of $\{\boldsymbol{z}_{b,n}^{\mathrm{em}}\}_{n=1}^{L}$, such as the arithmetic mean over pulse positions after Equation (4.5). For each b, collect the two pooled view embeddings $\bar{\boldsymbol{z}}_{b,(1)}^{\mathrm{em}}$ and $\bar{\boldsymbol{z}}_{b,(2)}^{\mathrm{em}}$ and form the index set $\mathcal{I}_b^{\mathrm{em}} = \{(b,1),(b,2)\}$ together with the positive pairing that swaps the two views. Concatenate negatives by including the first-view pooled embeddings of all *other* windows in the batch, following the standard "one positive, many negatives" recipe [4]. The emitter loss is the batch average of per-window

NT-Xent values,

$$\mathcal{L}_{\text{em}} = \frac{1}{B} \sum_{b=1}^{B} \mathcal{L}_{\text{NT}}\Big(\mathcal{I}_b^{\text{em}} \cup \mathcal{N}_b^{\text{em}},\, p(\cdot)\Big). \tag{4.8}$$

where $\mathcal{N}_b^{\text{em}}$ denotes the chosen negative multiset for window b. This mean over windows matches the implementation choice to score each window's contrastive problem separately before aggregating, which keeps the repulsive set size stable when B changes.

Mode-oriented term

Operating modes induce finer structure within an emitter, often visible only when comparing pulse patterns across longer horizons than a single position index. The mode branch therefore uses the same NT-Xent functional form but mixes *intra-window* comparisons with *inter-window* comparisons so gradients flow both along the token sequence and between windows drawn from the same batch.

Intra-window mode contrast follows Equation (4.7) with $\mathcal{I}$ equal to pulse indices inside a window, positives given by the second augmented view at the same token index, and negatives taken from other positions in the same window and view, which encourages local discrimination of mode-relevant features.

Inter-window mode contrast forms ordered pairs (b, b') of distinct batch indices (for example random pairs or pairs chosen to exploit the half-window stride in Section 4.2 when two windows overlap in time). For each pair, embeddings $\boldsymbol{z}_{b,n}^{\text{md}}$ and $\boldsymbol{z}_{b',n'}^{\text{md}}$ are compared for a small set of index pairs (n, n') (including the diagonal $n = n'$ if both windows are aligned to the same clock after preprocessing). The resulting NT-Xent terms are averaged over sampled pairs and, together with the intra-window term, define $\mathcal{L}_{\text{md}}$. The construction parallels visual pipelines in which two "crops" of the same image are positives [4], except that the natural pairing is induced by overlapping windows and augmentation rather than by spatial cropping.

Combined objective and use of the projection heads

The overall training loss is the weighted sum

$$\mathcal{L} = \mathcal{L}_{\text{em}} + \lambda_{\text{md}}\, \mathcal{L}_{\text{md}}, \qquad \lambda_{\text{md}} > 0, \tag{4.9}$$

with λ_{md} treated as a hyperparameter and reported in Chapter 5. An auxiliary DEC-style KL term between soft cluster assignments and a target distribution [7] is a natural extension when trainable centroids or prototypes are added; the experiments chapter states whether such a term is active.

Following common practice in self-supervised encoders [4], the mode embeddings retained for visualization or nearest-neighbor mode analysis are the vectors $\boldsymbol{z}_n^{\text{md}}$ from Equation (4.5), that is, the outputs of the mode linear head. For emitter-centric deinterleaving, the downstream deliverable is primarily the discrete per-pulse assignment

$\hat{e}_n$ obtained by clustering pooled emitter embeddings or by reading off a discrete head if one is added later; once those labels are fixed, storing every $\boldsymbol{z}_n^{\mathrm{em}}$ is optional because the continuous representation is not required for the same operational tasks that consume hard emitter tracks.

4.5 Training Procedure

The parameter vector $\boldsymbol{\theta}$ attached to the maps in Equations (4.4) and (4.5) is estimated by minimising the training objective $\mathcal{L}$ in Equation (4.9) on mini-batches of windows. Gradients are obtained by reverse-mode automatic differentiation and used in an adaptive first-order optimiser of the Adam family [**kingma2015adam**]. Unless Chapter 5 specifies otherwise, the implementation follows the default moment decay rates $(\beta_1, \beta_2) = (0.9, 0.999)$, stabiliser $\epsilon = 10^{-8}$, and bias-corrected running estimates of the first and second moments described by **kingma2015adam**, as exposed by the PyTorch optimiser interface.

The learning rate follows a cosine decay from a peak value down to a small floor over the course of training, which keeps step sizes comparatively large while the loss landscape is coarse and shrinks them during the final refinement phase. Peak learning rate, weight decay, batch size, number of passes over the training windows, and the exact step counting convention (per epoch versus global step) are hyperparameters and are listed in Chapter 5. At the start of each epoch, training windows are shuffled so that consecutive mini-batches are not ordered by scenario; any fixed random seeds, data-loader worker counts, and reproducibility settings used in the reported runs are stated alongside those hyperparameters.

Dropout inside the transformer blocks of Section 4.3 follows the usual pattern of stochastic attention and feed-forward masks; dropout rates belong to the same hyperparameter table. Training can be stopped early when a validation score computed on held-out windows (for example the smoothed mini-batch loss or an auxiliary clustering criterion) fails to improve for a fixed patience measured in epochs, in which case the best checkpoint prior to stagnation is retained; patience, validation metric, and checkpoint policy are also recorded in Chapter 5.

If a DEC-style Kullback–Leibler term is added as suggested in Section 4.4, target distributions are recomputed from the student assignments on a schedule analogous to Xie et al. [7], and any sharpening temperature or update period is treated as part of the optimisation protocol reported in Chapter 5. Likewise, the mode-branch weight λ_{md} may be held small during an initial warm-up segment so that the trunk learns coarse temporal structure before the finer mode contrastive gradients dominate; whether such a ramp is used, and its length, is part of the same configuration table. The experiments in this thesis use a single continuous training phase; staged schedules such as representation pretraining followed by clustering fine-tuning would be documented here if they were adopted.

@articlekingma2015adam, title = Adam: A Method for Stochastic Optimization, author = Kingma, Diederik P. and Ba, Jimmy, journal = International Conference on Learning Representations (ICLR), year = 2015,

4.6 Evaluation Metrics

Unsupervised models yield continuous embeddings and discrete cluster assignments; external evaluation compares those assignments to reference emitter and mode labels whenever they exist (for example in simulated or curated benchmarks). This section defines the metrics reported later in Chapter 5 and clarifies what each quantity measures.

Information-theoretic agreement. NMI quantifies shared information between two partitions while normalizing by their marginal entropies, so that $0 \leq \text{NMI} \leq 1$, with larger scores indicating closer agreement [8]. AMI applies an additional correction for chance in the spirit of the adjusted Rand construction, which can stabilize comparisons when cluster sizes are imbalanced or the number of clusters differs from the number of reference classes [8]. Both treat the partitions as categorical fields over pulses (or over sequences, if labels are defined at sequence level) and are invariant to permutations of cluster indices.

Pairwise concordance. ARI compares all pulse pairs and measures how often predicted co-assignment matches the reference, while subtracting the expected agreement under a random partition with the same cluster sizes [9]. The score satisfies $\text{ARI} \leq 1$, with $\text{ARI} = 1$ for perfect agreement and values near zero under chance-like assignments; it is less sensitive than raw Rand index to homogeneous splits that inflate agreement counts.

Homogeneity and completeness. V-measure is the harmonic mean of homogeneity (each cluster contains members of a single reference class) and completeness (all members of a reference class fall in one cluster), again in $[0, 1]$ with 1 indicating perfect correspondence up to renaming [10]. It complements NMI and AMI by making the trade-off between fragmentation and merging explicit.

Hungarian accuracy. ACC denotes clustering accuracy after an optimal permutation of cluster indices is chosen by the Hungarian algorithm so as to maximize the number of correctly assigned points; we denote the resulting score by ACC. It is intuitive and widely reported, but it assumes one-to-one alignment between inferred clusters and reference classes and can be optimistic when the inferred number of clusters differs from the ground truth or when several physical modes map to the same cluster.

Purity. Purity averages, over predicted clusters, the largest fraction of points that share a single reference label. It is easy to interpret but favors fine partitions: splitting a coherent class into many singleton clusters yields high purity even though the clustering is not useful, so it is reported alongside the other metrics rather than in isolation.

Qualitative inspection. Two-dimensional scatter plots of embeddings after UMAP reduction [11] provide a sanity check that local neighborhoods respect emitter or mode structure, especially when quantitative scores are similar across methods. Such plots are illustrative only: distances in the projection need not match distances in the original embedding space.

Protocol remarks. Metrics are computed on held-out batches or splits reserved for testing; exact batch construction, scenario identifiers, and the relationship between scenario difficulty (for example, emitter count and overlap) and scores are documented with the experimental results in Chapter 5.

4.7 Baseline Methods

The baselines isolate the contribution of the dual-branch layout in Section 4.3 and of the mode-oriented term $\mathcal{L}_{\text{md}}$ in Equation (4.9) relative to a single-stack encoder and to classical clustering without a sequence model.

The first neural baseline removes the emitter and mode branches and replaces them with one stack of eight transformer-encoder layers of the same functional form as the shared trunk (hidden width, head count, and FFN expansion match that trunk so the comparison stresses topology and objective rather than a change in per-layer width). Token embeddings $\boldsymbol{h}_n^{(0)}$ from Equation (4.4) pass through this stack; a single linear head then maps each contextualised token to $\boldsymbol{z}_n \in \mathbb{R}^p$, using the same output dimension p as the emitter branch in the full model. Training minimises only the emitter-oriented contrastive loss $\mathcal{L}_{\text{em}}$ in Equation (4.8); $\mathcal{L}_{\text{md}}$ and its inter-window structure are omitted, so no auxiliary geometry is trained for operating modes. This configuration tests whether one representation, optimised purely for window-level deinterleaving, suffices for both emitter- and mode-level clusterings reported in Chapter 5.

The second baseline is HDBSCAN [3], a hierarchical density-based method that does not estimate a transformer. It clusters fixed-length vectors built deterministically from the per-feature scaled PDW windows in Section 4.2, for example by per-channel summary statistics or a flattened layout of pulses and features; the exact vectorisation is fixed in Chapter 5. HDBSCAN separates gains due to learned sequence context from what density-based Euclidean clustering already recovers on hand-specified summaries.

Further comparisons—k-means on raw window vectors or on autoencoder embeddings, DEC-style objectives when centroids are used, SwAV-style assignment prediction

[6], and a supervised classifier upper bound when labels are available—are listed with their configurations in Chapter 5.

CHAPTER 5

Experiments and Results

5.1 Experimental Setup

Placeholder paragraph.

5.2 Datasets

Placeholder paragraph.

5.3 Hyperparameters and Training Details

Placeholder paragraph.

5.4 Quantitative Results

Placeholder paragraph.

5.5 Ablation Studies

Placeholder paragraph.

5.6 Qualitative Analysis

Placeholder paragraph.

CHAPTER 6

Discussion

6.1 Interpretation of Results

Placeholder paragraph.

6.2 Comparison with Related Work

Placeholder paragraph.

6.3 Limitations

Placeholder paragraph.

6.4 Implications

Placeholder paragraph.

CHAPTER 7

Conclusion

7.1 Summary

Placeholder paragraph.

7.2 Answers to the Research Questions

Placeholder paragraph.

7.3 Future Work

Placeholder paragraph.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[2] E. Gunn, A. Hosford, D. Mannion, J. Williams, V. Chhabra, and V. Nockles, *Radar pulse deinterleaving with transformer based deep metric learning*, arXiv preprint arXiv:2503.13476, 2025. arXiv: `2503.13476 [cs.LG]`.

[3] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering", *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017. DOI: `10.21105/joss.00205`

[4] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations", in *International Conference on Machine Learning (ICML)*, 2020.

[5] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, arXiv preprint arXiv:1807.03748, 2018. arXiv: `1807.03748 [cs.LG]`.

[6] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, "Unsupervised learning of visual features by contrasting cluster assignments", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis", in *International Conference on Machine Learning (ICML)*, 2016, pp. 478–487.

[8] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance", *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[9] L. Hubert and P. Arabie, "Comparing partitions", *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[10] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure", in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 2007, pp. 410–420.

[11] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction", *arXiv preprint arXiv:1802.03426*, 2018.

APPENDIX A

Additional Results

Placeholder paragraph.

APPENDIX B

Implementation Details

Placeholder paragraph.

APPENDIX C

Notation Reference

Placeholder paragraph.